

העברת רשומת DNS ל-Load Balancer הפעיל – הסבר

כיצד סקריפט ה-failover של SolarWinds מחליט לאיזה מרכז נתונים להפנות את הרשומה, ומה נדרש כדי שההחלטה הזו תהיה נכונה גם כשדברים נשברים.

תוכן העניינים

1. מה האוטומציה הזו עושה
2. הארכיטקטורה: שני מרכזי נתונים ורשומה אחת
3. מבנה הקבצים
4. זרימת הסקריפט, שלב אחר שלב
5. הפונקציות בפירוט
6. לוגיקת ההחלטה – טבלת אמת
7. מה תוקן מול הגרסה המקורית
8. פרמטרים
9. הפעלה, לוגים וקודי יציאה
10. מה לבדוק לפני הרצה אמיתית

1. מה האוטומציה הזו עושה

ל-SolarWinds יש שם DNS אחד שכולם בארגון משתמשים בו, למשל `OurSolar.OurZone`. השם הזה חייב להצביע תמיד על `Load Balancer` שבאמת מסוגל להגיש את הממשק. יש שני מרכזי נתונים, לכל אחד LB משלו וזוג שרתי web מאחוריו, והשאלה שהסקריפט עונה עליה היא אחת: **לאיזה מהשניים הרשומה צריכה להצביע עכשיו?** הסקריפט מופעל עבור מרכז הנתונים שהפך לפעיל, בודק דרך SNMP את שרתי ה-web של אותו מרכז, ומעדכן את רשומת ה-A בהתאם:

- יש לפחות שרת web אחד למעלה במרכז הנתונים הפעיל → הרשומה מצביעה על ה-LB המקומי.
- כל שרתי ה-web שם נפלו → הרשומה עוברת ל-LB של מרכז הנתונים השני.
- הרשומה כבר מצביעה למקום הנכון → לא נכתב דבר.

התנאי השלישי הוא מה שהופך את הסקריפט לבטוח להרצה חוזרת: הרצה שאין בה מה לשנות אינה כותבת ל-DNS כלל, ולכן אפשר להריץ אותו מתוזמן בלי לייצר תנועת כתיבה מיותרת או רעש בלוגים.

2. הארכיטקטורה: שני מרכזי נתונים ורשומה אחת

רכיב	PrimaryDC	SecondaryDC	תפקיד
שרת SolarWinds	10.10.10.1	10.10.20.1	מי שמריץ את הסקריפט. הכתובת שלו מועברת כפרמטר ActiveServer - וקובעת עבור איזה מרכז נתונים הריצה מתבצעת.
Load Balancer	10.10.10.10	10.10.20.10	היעד של רשומת ה-DNS. אחד משני אלה ייכתב לרשומה.
כתובת ניהול SNMP	10.10.10.100	10.10.20.100	הכתובת שנשאלת על מצב שרתי ה-web. זו כתובת הניהול של ה-LB, לא כתובת השירות.
רשומת DNS	OurSolar.OurZone		רשומת A יחידה. זו הרשומה שהסקריפט משנה.

שימו לב להבחנה בין שתי הכתובות של ה-LB. כתובת השירות (10) היא מה שנכתב לרשומת ה-DNS, וכתובת הניהול (100) היא מה שנשאל ב-SNMP. אלה שני דברים נפרדים ואסור לבלבל ביניהם: החלפת אחת בשנייה תייצר סקריפט שנראה תקין אבל מפנה תעבורה לכתובת שאינה מגישה את השירות.

ה-OIDים של SNMP

הערכים הנשאלים הם ה-OIDים תחת 1.3.6.1.4.1.1872, שהוא מרחב ה-enterprise של Radware. הספרות בסוף אינן שרירותיות – הן מקודדות את שמות השרתים כתווי ASCII. לדוגמה, ב-PrimaryDC הסיומת 52.52 היא "44" והסיומות 57.51 ו-57.52 הן "93" ו-"94".

```
# PrimaryDC - two web servers
.1.3.6.1.4.1.1872.2.5.4.2.19.1.10.2.52.52.1.2.57.51
.1.3.6.1.4.1.1872.2.5.4.2.19.1.10.2.52.52.1.2.57.52

# SecondaryDC - two web servers
.1.3.6.1.4.1.1872.2.5.4.2.19.1.10.2.51.57.1.2.51.54
.1.3.6.1.4.1.1872.2.5.4.2.19.1.10.2.51.57.1.2.51.55
```

המשמעות המעשית: ה-OIDים תלויים בשמות שהוגדרו על ה-LB. אם שרת web מוחלף או משנה שם, ה-OID משתנה איתו והסקריפט יתחיל לקרוא ערך של שרת שאינו קיים. לכן הם פרמטרים (PrimaryDcWebServerOid -, SecondaryDcWebServerOid -) ולא קבועים בקוד.

3. מבנה הקבצים

קובץ	תפקיד
DNS.SetSolarWindsRecordToActiveLB.ps1	הגרסה המקורית, כפי שתומללה מההדפסה. נשמרה כפי שהיא לצורך השוואה בלבד.
DNS.SetSolarWindsRecordToActiveLB.v2.ps1	הגרסה המתוקנת. אותה החלטה בדיוק, אבל עם טיפול בשגיאות, לוג, ולידציה של פרמטרים ותמיכה ב-WhatIf.

שני הקבצים מיישמים את אותו כלל עסקי. ההבדל אינו במה שהם מחליטים אלא במה קורה כשמשהו במסלול נשבר – וזה בדיוק המצב שבו סקריפט failover נדרש לעבוד.

4. זרימת הסקריפט, שלב אחר שלב

1. **זיהוי מרכז הנתונים.** הפרמטר `ActiveServer` מושווה לשתי כתובות שרתי ה-SolarWinds. ההתאמה קובעת מיהו ה-LB המקומי, מיהו המרוחק, ואיזו כתובת SNMP לשאול. כתובת שאינה מתאימה לאף אחת מהשתיים עוצרת את הריצה בשגיאה.
2. **איתור שרת DNS.** אם לא הועבר `DnsServer`, הסקריפט עובר על שרתי ה-DNS המוגדרים על כרטיסי הרשת ובוחר את הראשון שעונה ל-ping.
3. **קריאת המצב הנוכחי.** רשומת ה-A נקראת, והכתובת שהיא מצביעה עליה נשמרת להשוואה.
4. **תשאול SNMP.** כל OID נקרא, מספר פעמים לפי `PollCount`. ברגע שנמצא שרת אחד למעלה, התשאול נעצר ומרכז הנתונים מסווג כ-`Serving`.
5. **בחירת היעד.** `Serving` → ה-LB המקומי. `Down` → ה-LB המרוחק.
6. **כתיבה או דילוג.** אם הרשומה כבר מצביעה על היעד, הריצה מסתיימת בלי כתיבה. אחרת הרשומה מעודכנת, ומיד לאחר מכן נקראת שוב כדי לוודא שהערך אכן נשמר.

5. הפונקציות בפירוט

Resolve-DnsServerAddress

מחזירה את כתובת שרת ה-DNS הראשון שעונה. המסנן `IPEnabled = True` חשוב: בלעדיו כרטיסי רשת מנוטרלים או מנותקים תורמים כתובות DNS ישנות לרשימה, והסקריפט עלול לנסות לכתוב לשרת שכבר אינו רלוונטי.

```
$adapters = Get-CimInstance -ClassName Win32_NetworkAdapterConfiguration `
    -Filter 'IPEnabled = True'

foreach ($adapter in $adapters) {
    foreach ($address in $adapter.DNSServerSearchOrder) {
        $answered = Test-Connection -ComputerName $address -Count 1 `
            -Quiet -ErrorAction SilentlyContinue

        if ($answered) {
            return [string]$address
        }
    }
}

throw 'None of the DNS servers configured on this host answered a ping.'
```

זה בדיוק המקום שבו הגרסה המקורית נשברה. שם, הפקודה `Test-Connection` נקראה בלי שהפלט שלה נתפס, ולכן הפלט הזה **דלף** לערך ההחזרה של הפונקציה. במקום מחרוזת, הפונקציה החזירה מערך של שני איברים: אובייקט ה-ping ואחריו הכתובת. השורה שאחריה, `$dns_server_ip.IPv4Address.IPAddressToString`, עבדה רק בזכות הדליפה הזו – היא שלפה את הערך מאובייקט ה-ping. ב-PowerShell 7 המבנה של האובייקט הזה שונה, אין בו `IPv4Address`, והמשתנה היה מתאפס ל-`$null`.

Get-WebServerHealth

פותחת session אחד של SNMP, קוראת את כל ה-OID-ים דרכו, וסוגרת אותו. מחזירה שני שדות: `Answered` – האם התקבלה תשובה כלשהי, ו-`Healthy` – כמה שרתים דיווחו על עצמם כתקינים.

ההפרדה בין שני השדות היא העיקר. "כל השרתים נפלו" ו"לא הצלחנו לשאול" הם שני מצבים שונים לחלוטין, והם מובילים להחלטות שונות. בגרסה המקורית שניהם נראו זהים.

```
try {
    $snmp = New-Object -ComObject olePrn.oleSNMP
    $snmp.Open($SnmpHost, $SnmpCommunity, 2, 1000)

    foreach ($singleOid in $Oid) {
        try { $raw = ([string]$snmp.Get($singleOid)).Trim() }
        catch { Write-Log "..." -Level WARN; continue }

        $answered++
        if (-not [int]::TryParse($raw, [ref]$value)) { continue }
        if ($value -eq $HealthyValue) { $healthy++ }
    }
}
catch { Write-Log "SNMP session to $Label failed: ..." -Level WARN }
finally { if ($snmp) { $snmp.Close(); ... } }
```

שלוש נקודות בקטע הזה:

- ה-`try/catch` החיצוני תופס כשל בפתיחת ה-session. **בגרסה המקורית לא היה כזה**, ולכן LB שאינו זמין הפיל את הסקריפט כולו – כלומר בדיוק בתרחיש שבשבילו הסקריפט קיים, לא התבצע failover ואיש לא ידע.
- ה-`finally` מבטיח סגירה ושחרור של אובייקט ה-COM גם במסלול הכושל.
- `[int]::TryParse` במקום השוואה ישירה. `$snmp.Get()` מחזיר מחרוזת, והשוואה של "2" מול 0 עבדה במקרה – אבל רווח מיותר בפלט היה שובר אותה בשקט.

Get-DataCentreState

עוטפת את הקודמת בלולאת תשאול ומחזירה אחד משלושה מצבים:

מצב	מתי	ההשלכה
Serving	לפחות שרת אחד דיווח תקין, באחת מהבדיקות	הרשומה מצביעה על ה-LB המקומי
Down	התקבלו תשובות, וכולן מדווחות על שרתים שנפלו	failover ל-LB המרוחק
Unknown	אף בדיקה לא קיבלה תשובה כלשהי	נקבע לפי <code>-SnmpFailureMode</code>

הלולאה היא הגנת ה-flapping. בדיקת SNMP אחת שנכשלה רגעית לא מזיזה רשומת DNS; נדרשות `-PollCount` בדיקות רצופות (ברירת מחדל 3) לפני שמרכז נתונים מוכרז כמת. מנגד, תשובה חיובית אחת מספיקה כדי לעצור מיד – אין סיבה להמשיך לשאול כששרת נמצא למעלה.

Set-SolarWindsRecord ו-Get-SolarWindsRecord

הקריאה מבקשת `-RRType A` במפורש, ומסרבת לפעול אם השם מחזיר יותר מרשומה אחת:

```
$records = @(Get-DnsServerResourceRecord -ComputerName $Server -ZoneName $ZoneName `
    -Name $RecordName -RRType A)

if ($records.Count -eq 0) { throw "... holds no A record named '$RecordName'." }
if ($records.Count -gt 1) { throw "... holds $($records.Count) A records ..." }
```

בלי `-RRType A`, שם עם כמה רשומות (round-robin, למשל) היה מחזיר מערך, וההשמה `$NewObj.RecordData.IPv4Address = ...` הייתה נכשלת בהודעה לא ברורה. עדיף לסרב במפורש מאשר לנחש איזו רשומה לעדכן.

הכתיבה עצמה דורשת שני אובייקטים נפרדים – המקורי והמעודכן – ולכן נעשה `Clone()`. בסיום נקראת הרשומה שוב לאימות:

```

$current = Get-SolarWindsRecord -Server $Server
$updated = $current.Clone()
$updated.RecordData.Ipv4Address = [System.Net.IPAddress]::Parse($TargetIp)

if ($RecordTtlSeconds -gt 0) {
    $updated.TimeToLive = [System.TimeSpan]::FromSeconds($RecordTtlSeconds)
}

Set-DnsServerResourceRecord -ComputerName $Server -ZoneName $ZoneName `
    -OldInputObject $current -NewInputObject $updated

$after = Get-SolarWindsRecord -Server $Server
if ($after.RecordData.Ipv4Address.IPAddressToString -ne $TargetIp) {
    throw "The record was written but still reads ... instead of $TargetIp."
}

```

6. לוגיקת ההחלטה – טבלת אמת

הטבלה הבאה היא הליבה של הסקריפט. שימו לב שהכלל **זהה לחלוטין** בשני מרכזי הנתונים – אין כאן לוגיקה הפוכה, רק "מקומי" ו"מרוחק" שמתחלפים בזהותם:

מצב שרתי ה-web במרכז הפעיל	היעד הנבחר	נכתב ל-DNS?
שניהם למעלה	ה-LB המקומי	רק אם הרשומה עדיין לא שם
אחד למעלה, אחד נפל	ה-LB המקומי	רק אם הרשומה עדיין לא שם
שניהם נפלו	ה-LB המרוחק	רק אם הרשומה עדיין לא שם
SNMP לא ענה, מצב Down	ה-LB המרוחק	כן, עם אזהרה בלוג
SNMP לא ענה, מצב Unknown	אין שינוי	לא, יציאה בקוד 2

הנחת היסוד שחייבת אימות: הערך 0 מ-SNMP פירושו "השרת למעלה".

זהו הערך שהגרסה המקורית השוותה מולו, והוא נשמר כברירת מחדל. אם בפועל 0 אינו מסמן שרת תקין על ה-LB שלכם, **כל ההחלטות בטבלה מתהפכות** – הסקריפט יבצע failover כששרתים תקינים ויישאר במקום כשהם נפלו. זו ההנחה היחידה במסמך שאי אפשר לאמת מקריאת הקוד, וצריך לבדוק אותה מול הציוד.

הפרמטר **HealthyValue** – קיים בדיוק בשביל זה: אפשר לשנות את הערך בלי לגעת בקוד.

7. מה תוקן מול הגרסה המקורית

הבעיה	מדוע זה משנה	הפתרון
הפונקציה <code>dns_ip</code> נשענה על דליפת פלט	עבד רק ב-Windows PowerShell 5.1; ב-PowerShell 7 המשתנה היה מתאפס והכול היה נכשל	החזרת מחרוזת נקייה, בלי תלות בפלט הצדדי של <code>Test-Connection</code>
ה- <code>try/catch</code> סביב ה-ping מעולם לא נתפס, וה- <code>return</code> רץ תמיד	שרת ה-DNS הראשון ברשימה נבחר תמיד, גם אם היה מת. הלולאה כולה הייתה חסרת משמעות	<code>Test-Connection -Quiet</code> בתוך <code>if</code> ; שגיאה מפורשת אם אף שרת לא עונה
לא היה טיפול בשגיאות סביב SNMP	LB שאינו זמין הפיל את הסקריפט – כלומר לא בוצע failover בדיוק בתרחיש שלשמו נכתב	<code>try/catch/finally</code> מלא; אי-מענה הוא החלטה מנוהלת, לא קריסה
לא נדרש <code>RRType A</code>	שם עם כמה רשומות שבר את ההשמה בהודעה לא ברורה	סינון מפורש וסירוב מנומק במקרה של כפילות
בדיקה בודדת קבעה גורל	תקלת רשת רגעית הזיזה רשומת DNS	<code>PollCount</code> - בדיקות לפני הכרזה על נפילה
השוואת מחרוזת למספר	רווח מיותר בפלט היה שובר את ההשוואה בשקט	<code>[int]::TryParse</code> ; ערך לא-מספרי נספר כנפילה ונרשם בלוג
<code>Write-Host aaaaa</code> ארבע פעמים	אין תיעוד של מתי ולמה הרשומה הוחלפה	לוג עם חותמות זמן לקובץ ולמסך
ארבעה בלוקים כמעט זהים	כל תיקון היה צריך להיעשות ארבע פעמים	נתיב קוד אחד, עם פרמטרים לכיוון
<code>\$args[0]</code> בלי ולידציה	פרמטר שגוי או ריק סיים את הריצה בהצלחה מדומה, בלי לעשות דבר	<code>param()</code> עם <code>Mandatory</code> , <code>ValidateSet</code> ו- <code>ValidateRange</code>

8. פרמטרים

פרמטר	ברירת מחדל	מה הוא עושה
-ActiveServer	חובה	כתובת שרת ה-SolarWinds שעבורו הריצה מתבצעת
-DnsServer	איתור אוטומטי	שרת ה-DNS לקריאה וכתובה. חייב להחזיק עותק בר-כתיבה של ה-Zone
-HealthyValue	0	הערך שמסמן שרת תקין. ראו האזהרה בפרק 6
-PollCount	3	כמה בדיקות לפני הכרזה על נפילה
-PollDelaySeconds	5	השהיה בין בדיקות
-SnmpFailureMode	Down	Down – אי-מענה נחשב נפילה ומפעיל failover. Unknown – הרשומה נשארת במקומה והריצה מסתיימת בקוד 2
-RecordTtlSeconds	0	מעל אפס, קובע TTL חדש לרשומה בזמן הכתיבה
-LogPath	%ProgramData%\SolarWinds\dns-failover.log	קובץ הלוג
-WhatIf	–	מציג מה היה משתנה בלי לגעת ב-DNS. הלוג נכתב גם במצב זה

9. הפעלה, לוגים וקודי יציאה

```
powershell.exe -NoProfile -File DNS.SetSolarWindsRecordToActiveLB.v2.ps1 -ActiveServer 10.10.10.1
```

חובה להפעיל עם -File, לא עם -Command. פרמטר שנכשל בוולידציה עוצר את הסקריפט לפני שגופו רץ, ורק -File מתרגם את הכישלון הזה לקוד יציאה 1. תחת -Command הקורא מקבל ערך \$LASTEXITCODE ישן – כלומר שגיאת הקלדה בפרמטרים תיראה כמו ריצה מוצלחת.

קוד	משמעות
0	הסתיים כראוי – הרשומה עודכנה, או שלא היה מה לעדכן
1	כשל. הסיבה רשומה בלוג ונכתבת גם ל-stderr
2	מצב מרכז הנתונים לא נקבע, והרשומה הושארה ללא שינוי

הלוג נכתב בשתי דרכים במקביל – לקובץ ולמסך – כך ש-SolarWinds תופס את הפלט ובמקביל נשמר תיעוד היסטורי. כל שורה נושאת חותמת זמן ורמה, ושורת ההחלטה נראית כך:


```
2026-09-22 16:24:09 [INFO ] PrimaryDC has 2 web server(s) up.
2026-09-22 16:24:09 [INFO ] Reprinted OurSolar.OurZone from 10.10.20.10
to 10.10.10.10, the PrimaryDC load balancer.
```

10. מה לבדוק לפני הרצה אמיתית

1. **לאמת את משמעות הערך 0.** זו הבדיקה הקריטית ביותר. `snmpwalk` על ה-OID-ים מול שרת תקין ומול שרת מושבת ייתן את התשובה חד-משמעית.
2. **לוודא ששרת ה-DNS הוא Primary של ה-Zone.** כתיבה לעותק שאינו בר-כתיבה תיכשל. במקרה של ספק עדיף להעביר `DnsServer` מפורש במקום להסתמך על האיתור האוטומטי.
3. **לבדוק את ה-TTL הנוכחי של הרשומה.** TTL של שעה פירושו ש-failover ייכנס לתוקף רק בעוד שעה. ערך של 30 עד 60 שניות מתאים לרשומה שנועדה להתחלף.
4. **להחליף את מחרוזת ה-SNMP community.** הערך `public` הוא ברירת מחדל, לא המלצה.
5. **להריץ פעם אחת עם WhatIf** ולקרוא את הלוג לפני שנותנים לסקריפט לכתוב ל-DNS ייצור.

מצב הבדיקות. הגרסה המתוקנת נבדקה מול 17 תרחישים באמצעות תחליפים לפקודות Windows: failover לשני הכיוונים, שרת בודד למעלה, רשומה שכבר נכונה, כשל SNMP בשני המצבים, רשומות כפולות, היעדר רשומה, ערך SNMP לא תקין, `WhatIf`, וכל בדיקות הוולידציה של הפרמטרים. כל 17 עברו.

הבדיקות אינן תחליף להרצה מבוקרת בסביבת מעבדה: הן מאמתות את הלוגיקה, לא את ההתנהגות מול ציוד Radware אמיתי ומול שרת DNS אמיתי.

מסמך זה מתעד את הסקריפטים שתחת `Scripts/` בריפוזיטורי: `DNS.SetSolarWindsRecordToActiveLB.ps1` (הגרסה המקורית) ו-`DNS.SetSolarWindsRecordToActiveLB.v2.ps1` (הגרסה המתוקנת). כתובות ה-IP, שם ה-Zone ושם הרשומה המופיעים כאן הם ערכי ברירת המחדל שבקוד, וניתן לשנות כל אחד מהם בפרמטר בלי לערוך את הסקריפט.